

TICEC 2026 · WORKSHOP TÉCNICO INTERNACIONAL

Más allá del WAF: Detección de SQLi, Observabilidad con eBPF y Remediación

Un enfoque de Purple Teaming en tiempo real para aplicaciones Cloud-Native y protección en tiempo de ejecución.



Equipo Purple Team



Duración: 120 Minutos



TICEC 2026

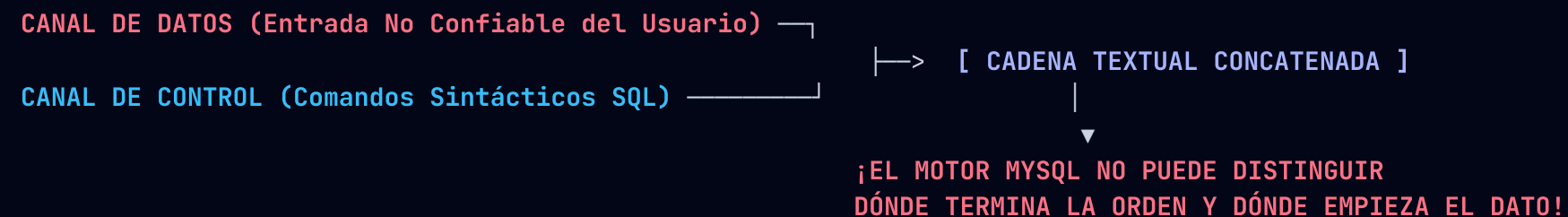
Ruta del Workshop (120 Minutos)

Del análisis de compiladores a las llamadas de Kernel y la Live Demo interactiva

01 25 min AST & La Falacia del WAF Anatomía del árbol sintáctico y por qué el perímetro de Capa 7 falla.	02 25 min Arquitectura Go & Concurrencia Worker Pool, Canales y sondas simultáneas (Boolean, UNION y Time).	03 20 min Observabilidad con eBPF Intercepción de Capa 0 con kprobes en socket 3306 y Tetragon Sigkill.	04 50 min DEMO EN VIVO & REMEDIACIÓN Ataque y defensa en tiempo real: Vulnerable vs Seguro + Q&A.
--	---	---	---

La Confusión de Canales (CWE-89)

¿Por qué sigue existiendo SQLi tras 25 años en el OWASP Top 10?



✗ Concatenación Dinámica

Al concatenar strings, la entrada del usuario es parseada directamente por el compilador de la base de datos como una instrucción ejecutable.

✓ Separación Física por Protocolo

Las sentencias preparadas compilan la gramática antes de recibir el dato. El valor viaja en paquetes binarios independientes.

Anatomía de la Alteración del AST

Cómo el analizador sintáctico del motor relacional reestructura la consulta

[CONSULTA ORIGINAL]

SELECT * FROM products WHERE id = [INPUT] AND is_active = TRUE;

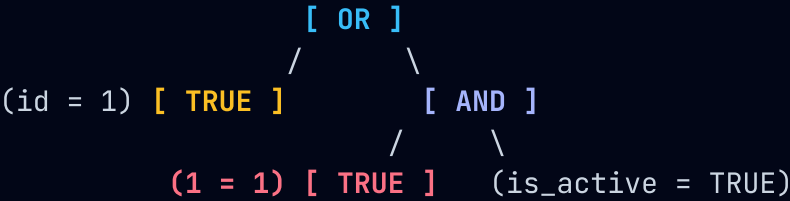
[ENTRADA INYECTADA]

INPUT: 1 OR 1=1

[CONSULTA FINAL]

SELECT * FROM products WHERE id = 1 OR 1=1 AND is_active = TRUE;

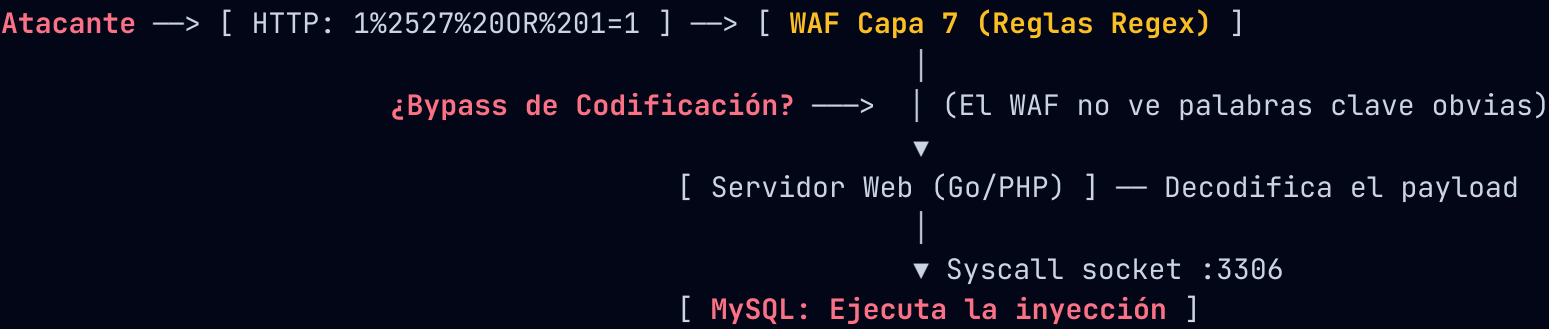
ÁRBOL SINTÁCTICO (AST) REESTRUCTURADO:



RESULTADO: Como (1=1) es una tautología absoluta, el WHERE evalúa a TRUE para toda la tabla.

La Falacia de la Seguridad en el Borde (WAF)

Por qué la inspección perimetral de Capa 7 es insuficiente por sí sola



Evasión por Encodings

Double URL encoding (`%2527`), secuencias multicarácter UTF-8 y comentarios en línea (`UNI/\*\*/ON`).

Ceguera de Contexto

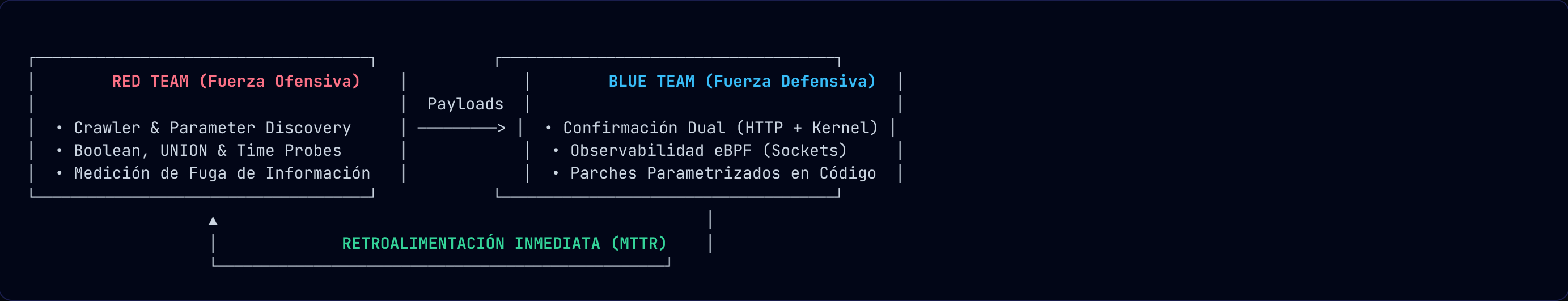
El WAF desconoce el esquema interno de base de datos y cómo el driver interpretará los bytes.

Falsos Positivos

Reglas regex estrictas terminan bloqueando búsquedas de usuarios legítimos con comillas o tildes.

El Paradigma Purple Team

Fusión continua de técnicas ofensivas y contramedidas defensivas



La Nueva Frontera: Observabilidad con eBPF

extended Berkeley Packet Filter: Visibilidad absoluta en el espacio de Kernel de Linux

```
ATACANTE —> [ HTTP Request: /products?id=1%20OR%201=1 ] (Capa 7)
      |
      ▼
    [ Servidor Web: vulnerable-app (:8081) ]
      |
      ▼ SYSCALL: sys_enter_write(fd=6, buf="SELECT ... WHERE id = 1 OR 1=1")
```

KERNEL DE LINUX (Espacio de Observabilidad eBPF)

Sensor kprobe intercepta el buffer transmitido hacia el socket TCP del puerto 3306:

```
"SELECT id, name, price FROM products WHERE id = 1 OR 1=1 AND is_active = TRUE"
```

```
      |
      ▼
    [ Motor MySQL 8.0 (:3306) ] —> Ejecución física del query
```

Runtime Security con Cilium Tetragon

Interrupción en tiempo real de llamadas al sistema sospechosas

TracingPolicy en Kubernetes

```
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: "block-sqli-runtime"
spec:
  kprobes:
    - call: "sys_enter_write"
      syscall: true
      selectors:
        - matchArgs:
            - index: 1
              operator: "Prefix"
              values:
                - "SELECT * FROM workshop_users"
                - "1 OR 1=1"
      matchActions:
        - action: Sigkill
```

Mecanismo de Interrupción

Cuando una consulta hacia el puerto 3306 coincide con un patrón anómalo no autorizado:

1. Hook en kprobe detecta el prefijo.
2. Tetragon genera evento de auditoría.
3. El Kernel despacha **SIGKILL** al hilo.
4. Conexión cerrada antes de procesar.

Arquitectura de Microservicios Contenerizados

Ecosistema modular aislado en redes virtuales privadas de Docker



Concurrencia en Go: El Patrón Worker Pool

Auditoría paralela de alto rendimiento sin riesgo de Denegación de Servicio (DoS)



Las 3 Sondas Simultáneas del Escáner

Cobertura integral de las principales variantes de explotación en CWE-89

1. Boolean Blind

Payload: `1 OR 1=1`
Métrica: Diferencial DOM
Efecto: WHERE evalúa TRUE.
Muestra productos ocultos.

2. UNION Extraction

Payload: `0 UNION SELECT...`
Métrica: Fuga de registros
Efecto: Concatena filas de
la tabla workshop_users.

3. Time-Based Blind

Payload: `1 AND SLEEP(2)`
Métrica: Diferencial Latencia
Base: ~15 ms
Inyectado: > 1800 ms.

Guardrails Éticos y Mitigación Anti-SSRF

Garantizando el uso académico responsable del software de seguridad

[Petición de Escaneo] —> [Policy Engine: ValidateTarget()]

▼

Modo Sandbox (Interno)

- Solo nombres registrados:
`vulnerable-app`, `secure-app`
- Rechazo de URLs arbitrarias

▼

Modo URL Externa (Pública)

- ¿Autorización confirmada? (Sí/No)
- ¿Esquema HTTP/HTTPS válido?
- Bloqueo de IPs numéricas (Anti-SSRF)
- Bloqueo de metadatos Cloud (169.254.x.x)
- Lista blanca: `testphp.vulnweb.com`

El Mecanismo de las Sentencias Preparadas

Por qué las Consultas Parametrizadas son 100% inmunes a la inyección

[FASE 1: PREPARE (Compilación del AST)]

El cliente envía la plantilla vacía:

```
"SELECT id, name, price FROM products WHERE id = ? AND is_active = TRUE;"
```

└─> El motor MySQL compila el AST. El signo '?' es fijado como un valor escalar literal.

[FASE 2: EXECUTE (Paso de Datos por Protocolo Binario)]

El cliente envía el parámetro:

Parámetro: "1 OR 1=1" (Tipo: String)

└─> MySQL busca productos cuyo id sea textualmente la cadena "1 OR 1=1".

└─> ¡IMPOSIBLE QUE EL DATO ALTERE LA LÓGICA DE LA CONSULTA PRECOMPILADA!

Vulnerable vs Seguro en los Sandboxes

Implementación real en los contenedores del taller

❌ vulnerable-app (Concatenación)

```
// Entrada contamina la gramática
query := "SELECT id, name FROM products " +
        "WHERE id = " + productID

// MySQL compila el texto alterado
rows, err := db.QueryContext(ctx, query)
```

Inyección exitosa: Boolean, UNION y SLEEP.

✅ secure-app (Defensa en Profundidad)

```
// 1. CPU valida tipo antes de tocar la BD
id, err := strconv.ParseUint(idText, 10, 64)

// 2. Parámetro posicional seguro
query := "SELECT id, name FROM products " +
        "WHERE id = ? AND is_active = TRUE"
row := db.QueryRowContext(ctx, query, id)
```

Resiliencia absoluta: 0 hallazgos.

LIVE DEMO

Plan de la Demostración Interactiva

Tres escenarios controlados ejecutados en vivo ante la audiencia

Escenario A: La Brecha

Target: vulnerable-app:8081. Ejecución paralela de sondas.
Alerta roja con confirmación dual eBPF.

Escenario B: La Resiliencia

Target: secure-app:8081. Mismos ataques contra código con
Prepared Statements. Alerta verde: 0 hallazgos.

Escenario C: Web Real

Target: testphp.vulnweb.com. Crawler dinámico de formularios y
parámetros GET externos.

LIVE DEMO · EN TIEMPO REAL

¡Pase a la Demostración Práctica!

Pasamos al Dashboard interactivo de Purple Teaming para ejecutar el escáner y visualizar la telemetría en vivo.



Abrir Dashboard (localhost:3000)

La Matriz de Confirmación Dual

Correlación matemática entre Capa 7 (HTTP) y Capa 0 (Kernel)

EVIDENCIA CAPA 7 (HTTP)	EVIDENCIA CAPA 0 (KERNEL eBPF)	VEREDICTO FINAL
• Diferencial de Longitud en DOM • Códigos de Error SQL en Body • Retardo temporal (> 1800 ms)	• Intercepción en socket :3306 • Buffer contiene sintaxis alterada • Syscall: sys_enter_write	INYECCIÓN CONFIRMADA (100% de Confianza) (Sin Falsos Positivos)
• Respuesta HTTP idéntica (200 OK) • Latencia normal (~15 ms)	• Socket recibe parámetro pre-enlazado • Sin alteración de gramática	SISTEMA RESILIENTE (Mitigación Acreditada)

Integración Continua: Shift-Left Security

Automatizando Purple Teaming y observabilidad eBPF en el pipeline CI/CD

```
[ Pull Request en Git ]
|
▼ (Disparo de Pipeline Automatizado en GitHub Actions)
[ Runner de CI/CD ]
├─ 1. SAST: Análisis estático buscando concatenaciones directas
├─ 2. Build Efímero: Despliegue de contenedores de prueba con Docker Compose
├─ 3. DAST Purple Scanner: Ejecución automatizada de sondas concurrentes
├─ 4. Quality Gate: Bloqueo del Merge si se confirma cualquier hallazgo CWE-89
|
▼ (Aprobado)
[ Despliegue a Kubernetes ] —> Protegido en Runtime con Políticas eBPF Tetragon (Sigkill)
```

TICEC 2026 · CONCLUSIONES

Conclusiones y Discusión Abierta

1. Diseño, no Parches

CWE-89 se soluciona separando el canal de datos del canal de control mediante sentencias preparadas.

2. El Kernel no Miente

eBPF elimina la ceguera de los WAFs de Capa 7, ofreciendo observabilidad y contención en Capa 0.

3. Sinergia Purple Team

La combinación de ataque automático con entrega de parches inmediatos reduce el tiempo de remediación a minutos.

 **Repo:** github.com/LeirBaGMC/SQLi-And-Recon-Tool

 **Stack:** Go 1.22 · React 19 · Docker · eBPF

 **Espacio para Preguntas (Q&A)**